Parts Discount Engine README

Parts Discount Engine

What This App Does

Parts Discount Engine is a garage pricing and deal-optimization web app.

It helps a sales user build a cart for a garage, apply the best safe discount, and discover the next best action to improve the deal.

The app answers questions like:

- What is the final price after discount?
- Which parts have better discount options?
- Which parts unlock extra discount at higher quantity?
- Which parts get extra discount when the cart reaches a value threshold?
- Which parts are commonly bought together?
- What should the user add to reach a target discount?

The UI is garage-facing, so it focuses on selling price, discount, final payable value, and quick add/switch actions. Internal margin is intentionally hidden from the visible line items.

Main User Flow

1. User opens the app.
2. App loads the parts catalogue from Google Sheets.
3. User builds a cart by selecting part, brand, and quantity.
4. User optionally enters the garage's target overall discount.
5. User clicks `Get Best Deal`.
6. Backend calculates item-level and cart-level pricing.

7. UI shows Deal Summary, Line Items, Smart Recommendations, To Reach Target Discount, and Bought Together.
8. User can add recommended parts directly with quantity controls.

Project Files

`app.py`

The Flask backend.

It loads Google Sheet data, calculates discounts, builds recommendations, and exposes API routes used by the frontend.

`index.html`

The page structure.

It contains the app header, Build Cart section, Bought Together section, Deal Summary, Line Items, Smart Recommendations, and To Reach Target Discount.

`script.js`

The frontend logic.

It loads data, manages cart rows, sends cart data to the backend, renders the results, handles add buttons, handles brand switching, and refreshes recommendation states.

`style.css`

The visual design.

It controls layout, cards, rows, badges, buttons, dropdowns, Bought Together rows, recommendation cards, and responsive behavior.

`requirements.txt`

Python dependencies needed for deployment.

`Procfile`

Deployment start command for platforms like Render and Railway.

Data Source

The app uses one Google Sheet with multiple tabs.

Sheet1: Catalogue

This is the parts master.

Important columns:

- `Part`
- `Brand`
- `SellingPrice`
- `BuyingPrice`
- `MinDiscount`
- `MinMarginPercent`
- `Stock`
- `QtyThreshold`
- `BonusDiscount_QtyThreshold`
- `MinCartValue`
- `BonusDiscount_MinCartValue`

Sheet2: Order History

Used for Bought Together recommendations.

Important columns:

- `Order Reference`
- `Quantity`
- `Part` or `Product Template/Name`

The backend supports both `Part` and `Product Template/Name`, so small column-name changes do not break recommendations.

Sheet3: Part Proxy Map

Used to group similar parts into logical recommendation families.

Important columns:

- `Part`
- `PartProxy` or `Part Proxy`

Example:

Different water paper variants can be grouped under `Water Paper / Regmar (Sandpaper)`.

Discount Logic

The backend calculates discount in layers.

1. Base Discount

`MinDiscount` is the starting discount for a part.

2. Quantity Bonus

If quantity reaches `QtyThreshold`, the app adds `BonusDiscount_QtyThreshold`.

Example:

- QtyThreshold = 6
- BonusDiscount_QtyThreshold = 3%
- User buys 6 or more
- App gives 3% extra discount on that item

3. Cart Value Bonus

If the total cart value reaches `MinCartValue`, eligible parts get `BonusDiscount_MinCartValue`.

Important product rule:

The user can add any parts to reach the threshold. The extra discount applies only to the eligible part.

Example:

- Coolant Green gets +10% at cart value Rs. 10,000
- User adds Coolant Green
- User adds any other parts until cart reaches Rs. 10,000
- Coolant Green receives the extra discount

4. Margin Guard

After applying discount, the backend checks minimum margin.

If discount makes margin unsafe, the backend reduces discount in 0.5% steps until margin is safe.

This means the app tries to give the best possible discount without breaking margin rules.

Main UI Sections

Build Cart

Users choose part, brand, and quantity.

The brand dropdown also shows selling price, so users can choose the correct variant.

Deal Summary

Shows:

- Total Cart Value
- Discount Given
- Value After Discount

Line Items

Shows each cart item with:

- Part
- Brand
- Quantity

- Unit Price
- Discount
- Final Price

Margin is not shown because garages should not see internal profitability.

Smart Recommendations

Shows actions that can improve the deal:

- better brand switch options
- cart-value discount parts
- direct add buttons with quantity controls

To Reach Target Discount

Appears when the user enters a target discount.

It shows:

- current effective discount
- gap to target
- quantity nudges
- brand switch nudges
- top 3 add-part suggestions that improve overall discount

Quantity nudges appear only here, so the same advice is not repeated in multiple sections.

Bought Together

Placed below Build Cart because it directly helps users add more relevant parts.

It recommends parts based on historical order baskets.

Recent UX improvement:

If many variants have the same part name, the UI now shows one row with:

```
Part Name | Brand + Price dropdown | Qty | Add
```

This saves space and makes the recommendation easier to use.

How Bought Together Works

Bought Together uses Sheet2 order history.

Simple explanation:

1. Backend reads all order rows.
2. It groups rows by `Order Reference`.
3. Each order becomes a basket of parts.
4. Parts are mapped to proxy families using Sheet3.
5. Backend counts how often two proxy families appear together.
6. It calculates relationship strength using co-order count, Jaccard, and Lift.
7. Frontend shows the strongest addable parts that are not already in the cart.

Co-order Count

How many orders had both Part A and Part B.

Jaccard

Of all orders that had either A or B, how many had both.

This tells us how much the two parts overlap.

Lift

How much more often A and B appear together compared to random chance.

This helps find strong pairings, not just common items.

Fallback Popular Add-ons

If direct pair data is missing, the app now fills Bought Together with popular order-history add-ons instead of showing a blank section.

This avoids the bad user experience of “No frequent bought-together parts found” when we still have useful popular parts to suggest.

Add Button Behavior

Every recommendation add button has:

- quantity input
- `+ Add` button

If the same part and brand already exist in cart, the app increases quantity instead of creating duplicate rows.

After adding, the app automatically recalculates the cart and refreshes:

- Deal Summary
- Line Items
- Smart Recommendations
- To Reach Target Discount
- Bought Together

If the user removes a part, the relevant recommendation button changes back from `Added` to `+ Add`.

Important Backend Functions

`_fetch(url)`

Loads CSV data from Google Sheets and caches it for speed.

`_normalise(row)`

Converts Sheet1 column names into backend-friendly keys.

Example:

`SellingPrice` becomes `selling_price`.

`_row_value(row, *names)`

Safely reads a value even if a sheet column name changes.

Example:

It can read either `PartProxy` or `Part Proxy`.

`resolve_proxy(part, proxy_map)`

Maps a part name to its proxy family.

It first tries exact match, then normalized match.

`compute_pricing(row, qty, cart_total_sp)`

Calculates discount and final price for one line item.

`evaluate_cart(cart, catalogue)`

Calculates the full cart.

`build_suggestions(cart, catalogue, result)`

Builds Smart Recommendations.

`build_target_advice(cart, catalogue, result, target)`

Builds To Reach Target Discount recommendations.

`compute_reco_map(top_n)`

Builds the Bought Together recommendation matrix from order history.

`get_cart_recos(cart_items, reco_map, proxy_map, catalogue)`

Returns the actual Bought Together recommendations for the current cart.

`_popular_proxy_recos(...)`

Fallback that returns popular add-ons if direct pair recommendations are not available.

Important Frontend Functions

`loadData()`

Loads catalogue data from `/api/data`.

`addItem(prefillPart, prefillBrand)`

Adds a row to Build Cart.

`updateBrands(row, prefillBrand)`

Updates brand options when part changes.

`getCart()`

Reads cart rows and merges duplicate part-brand rows.

`calculateCart()`

Sends cart to `/api/deal` and renders the response.

`renderRecommendations(recos)`

Renders Bought Together.

It groups same part names and uses a brand-price dropdown to save space.

`addOrIncrementCart(part, brand, qty, btn)`

Adds a recommended part or increases quantity if it already exists.

It also recalculates the cart after the add action.

`switchBrandInCart(part, fromBrand, toBrand, btn)`

Switches the brand in the matching cart row and recalculates the cart.

API Routes

`/`

Serves the main app page.

`/api/data`

Returns catalogue data for dropdowns.

`/api/deal`

Main calculation API.

Input:

```
{
  "cart": [
    { "part": "Coolant Green", "brand": "Golden Star", "qty": 1 }
  ],
  "target_discount": 10
}
```

Output:

- `result`
- `suggestions`
- `target_advice`
- `recommendations`

`/api/debug_sheets`

Debug route to check whether Google Sheet tabs are loading correctly.

`/api/reco_matrix`

Returns Bought Together recommendation matrix as JSON.

`/api/combinations_report`

Returns an Excel report from backend data.

The UI download button was removed, but the backend route still exists.

How To Run Locally

Open terminal in the project folder:

```
cd /Users/rajeevyadav/Documents/NEWFINALAPP
/opt/anaconda3/bin/python -B app.py
```

Then open:

```
http://127.0.0.1:3000
```

How To Put This App Live

The simplest option is Render because this is a small Flask app and the project now has `requirements.txt` and `Procfile`.

Option A: Deploy On Render

1. Create a GitHub repository.
2. Upload these files:
   - `app.py`
   - `index.html`
   - `script.js`
   - `style.css`
   - `requirements.txt`
   - `Procfile`
3. Go to Render.
4. Click `New +`.
5. Choose `Web Service`.
6. Connect your GitHub repository.
7. Use these settings:
   - Environment: Python
   - Build Command: `pip install -r requirements.txt`
   - Start Command: `gunicorn app:app`
8. Deploy.
9. Render will give you a public URL.
10. Share that URL with users.

Option B: Deploy On Railway

1. Create a GitHub repository with the same project files.
2. Go to Railway.
3. Create a new project from GitHub.
4. Railway should detect Python.

5. Set start command to:

```
gunicorn app:app
```

6. Deploy and share the generated public URL.

Option C: Deploy On PythonAnywhere

PythonAnywhere is also good for Flask, but setup is more manual.

Use it if you want a simpler Python hosting environment and are comfortable configuring WSGI.

Google Sheet Requirements For Live App

For the live app to work, the Google Sheet must be reachable by the server.

Best approach:

1. Keep the Google Sheet as `Anyone with the link can view`, or publish CSV access.
2. Do not put private cost-sensitive data in a public sheet unless access is controlled.
3. Keep Sheet tab GIDs stable.
4. If you duplicate or recreate tabs, update the GIDs in `app.py`.

Current code expects:

- Sheet1 for catalogue
- Sheet2 for order history
- Sheet3 for proxy map

Live-App Checklist Before Sharing

- Confirm `/api/data` loads parts.
- Confirm `/api/debug_sheets` shows rows for Sheet1, Sheet2, and Sheet3.
- Add a test cart and click `Get Best Deal`.
- Confirm Deal Summary appears.
- Confirm Bought Together has recommendations.
- Confirm Add buttons refresh the cart.
- Confirm garage-facing UI does not show margin.

- Confirm Google Sheet data does not expose anything you do not want users to see.

Recommended Future Improvements

Add SKU Or VariantID

This is the most important next improvement.

Right now, many operations identify products using:

```
Part + Brand
```

If the same part and same brand exist at different prices, that can be ambiguous.

A unique SKU or VariantID will make add, switch, recommendation tracking, and analytics much safer.

Add Admin View

A simple admin/debug page could show:

- cart bonus parts
- quantity bonus parts
- high discount parts
- stock issues
- missing proxy mappings

Track Recommendation Analytics

Useful events:

- recommendation shown
- recommendation added
- brand switched
- target discount achieved

This will help understand which nudges actually increase cart value.

Improve Bought Together With Business Rules

Order history is useful, but business rules can improve quality.

Examples:

- Paint workflow bundles
- consumable add-ons
- high-stock preferred products
- margin-safe priority products

One-Line Summary

Parts Discount Engine is a rules-plus-data recommendation engine that helps sales users build better garage carts by applying safe discounts, showing discount unlocks, recommending commonly bought parts, and guiding users toward target discounts.